Part A – Understanding Lua 5.5's Garbage Collector

Design Optimization of Computing Systems – Assignment 1

1 Objective and Experimental Setup

The objective of Part A is to understand the stock Lua 5.5 garbage collector and relate its source-level implementation to the Callgrind results. The main files studied were `lgc.c` and `lgc.h`, together with the table/object and allocation code used by the benchmark.

The benchmark executes $1000 \times 1000 = 1,000,000$ iterations. Each iteration creates an outer table and an inner table. Every 100th outer object is stored in `survivors`. Thus, most allocated objects are short-lived, while a smaller population remains reachable for longer.

Executable	Configuration	Wrapper operation
`fullgc`	Full	`lua_gc(L,LUA_GCCOLLECT)`
`incrementalgc`	Incremental	`lua_gc(L,LUA_GCINC)`
`generationalgc`	Generational	`lua_gc(L,LUA_GCGEN)`

Table 1: GC configurations used for Part A.

An important experimental detail is that `LUA_GCCOLLECT` performs a full collection when the call is made; it is not a permanent "full GC mode". Therefore, the `fullgc` workload profile is expected to be close to the incremental run.

2 Understanding the GC Source

Lua's collector can be viewed as a state machine. The main sequence is approximately

```
GCSpause -> GCSpropagate -> GCSenteratomic -> GCSatomic
         -> GCSswpallgc -> GCSswpfinobj -> GCSswptobefnz
         -> GCSswpend -> GCScallfin -> GCSpause
```

Lua uses tri-colour marking. Newly created objects start white. Reachable objects are marked and placed on gray lists when their references still need to be inspected. `propagatemark()` processes a gray object and follows its references, giving the basic transition

$$\text{WHITE} \rightarrow \text{GRAY} \rightarrow \text{BLACK}.$$

After ordinary propagation, `atomic()` completes the marking phase and handles roots and special cases such as weak tables, ephemerons, upvalues and finalizable objects. The sweep states then reclaim unreachable objects, while finalizer-related states handle finalizable objects.

`singlestep()` is useful for connecting the source to the measured behaviour because it examines `g->gcstate` and performs one unit of work for the current state. Incremental collection uses `incstep()` to repeatedly perform such small steps. Generational collection additionally uses object age and generation-aware paths such as `sweepgen()`.

3 Understanding the Important Call Counts

The large call counts observed in Callgrind can be explained from the benchmark itself. Approximately two million table objects are created, and the table assignments and allocation paths are therefore also executed on a similar scale.

Function	Calls	Reason for the count
`luaH_new`	2,000,022	Two tables per iteration.
`luaC_newobj`	2,000,259	GC object creation path.
`luaH_finishset`	2,000,041	Table assignment operations.
`luaH_newkey`	2,000,041	Table entry insertion/update.
`luaH_resize`	3,000,054	Table storage resizing.
`propagatemark`	1,017,556	Gray-object propagation.
`singlestep`	1,167,567	Repeated GC state-machine steps.
`freeobj`	2,000,259	Reclamation of allocated objects.

Table 2: Important call counts for the Full-GC execution.

The main call chains observed are:

```
luaV_execute -> luaH_new -> luaC_newobj -> luaC_newobjdt -> luaM_malloc_

luaV_execute -> luaV_finishset -> luaH_finishset
             -> luaH_newkey -> luaH_resize

luaC_step -> singlestep -> propagatemark

sweeplist -> freeobj -> luaH_free -> luaM_free_
```

These chains connect the benchmark operations to object creation, table updates, marking, and reclamation. In particular, the approximately two million calls to the table creation and reclamation functions are consistent with the two table objects created in every benchmark iteration.

4 Callgrind Instruction Analysis

Callgrind reports executed instructions using the `Ir` event. The inclusive instruction cost of the high-level function `luaC_step()` is used:

$$\text{GC instruction percentage} = \frac{\text{inclusive instructions of } \texttt{luaC_step}}{\text{total executed instructions}} \times 100.$$

Configuration	Total `Ir`	Approx. GC instructions	GC share
Full	3,710,041,444	873.7 M	23.55%
Incremental	3,700,133,914	868.1 M	23.46%
Generational	3,574,116,509	748.1 M	20.93%

Table 3: Callgrind instruction results for $m = 1000, n = 1000$.

The percentages of nested functions must not be added. For example, the costs of `luaC_step()`, `singlestep()`, and `propagatemark()` are inclusive, so the lower-level costs are already contained in their callers. Adding them would double-count instructions.

Therefore, the inclusive cost of `luaC_step()` is used as the high-level measure of GC instruction consumption.

5 Full GC Analysis

For Full GC, `luaC_step()` has 23.55% inclusive cost, with about 1.17 million `singlestep()` calls and 1.02 million `propagatemark()` calls. This shows that a significant amount of execution is spent on repeated GC state-machine work and reachability traversal.

The table side is also prominent: `luaH_finishset()` accounts for 40.46%, `luaH_newkey()` for 38.89%, and `luaH_resize()` for 23.59%. These high inclusive costs are consistent with the large number of table operations in the benchmark.

The `freeobj()` function has an inclusive cost of 16.44% and is called approximately 2.00 million times. This is consistent with the large population of temporary objects eventually being reclaimed.

6 Incremental GC Analysis

The Incremental results are almost identical to the Full-GC results in the important GC paths. `luaC_step()` accounts for 23.46% of instructions, compared with 23.55% for Full GC.

There are approximately 1.159 million `singlestep()` calls and 1.010 million `propagatemark()` calls. This behaviour is consistent with the incremental implementation using `incstep()` to repeatedly perform small units of GC work.

The similarity should not be interpreted as the two wrappers selecting exactly the same mode. The Full wrapper performs `LUA_GCCOLLECT` once before the benchmark. After this collection completes, the benchmark continues to execute normally. Hence, the measured workload profile can remain very close to the incremental behaviour.

The small difference between 23.55% and 23.46% is therefore expected for this experimental setup rather than evidence of a major behavioural difference between the two measured runs.

7 Generational GC Analysis

The Generational results are noticeably different. `luaC_step()` accounts for 20.93% of the executed instructions, which is lower than both Full GC and Incremental GC.

Only about 86,815 `singlestep()` calls occur, compared with more than 1.15 million in the other two configurations. Similarly, `propagatemark()` falls to about 148,334 calls and 3.15% inclusive cost.

A major generational path is `sweepgen()`, which accounts for 16.65% inclusive cost and approximately 13,495 calls. There are also about 1.88 million calls through the `sweepgen -> freeobj` path.

This behaviour matches the benchmark. Roughly two million table objects are created, but most of them become unreachable quickly. Only every 100th outer object is retained in `survivors`. The workload therefore contains a large young, short-lived population and a much smaller long-lived population.

Generational collection can make use of this lifetime information. As a result, the measured execution contains much less ordinary propagation and significantly fewer general state-machine steps, while generation-aware sweeping becomes an important part of the GC work.

This explains why the measured GC instruction share falls to 20.93%.

8 Comparison of the Three Configurations

Metric	Full	Incremental	Generational
Total `Ir`	3.710 B	3.700 B	3.574 B
GC instruction share	23.55%	23.46%	20.93%
`singlestep` calls	∼1.168 M	∼1.159 M	∼86.8 K
`propagatemark` calls	∼1.018 M	∼1.010 M	∼148 K
`sweepgen` inclusive cost	–	–	16.65%
`freeobj` inclusive cost	16.44%	16.48%	17.05%

Table 4: Summary of the main Callgrind results.

The most important variation is between the Generational configuration and the other two configurations. Full and Incremental GC consume almost the same fraction of instructions, at 23.55% and 23.46%, respectively.

Generational GC reduces this fraction to 20.93%. It also performs far fewer calls to `singlestep()` and `propagatemark()`, while introducing the generation-aware `sweepgen()` path.

Thus, the difference is not only visible in the final instruction percentage; it is also supported by the change in the functions responsible for the GC work.

9 Conclusion

For the original $m = 1000, n = 1000$ workload, the measured GC instruction shares are:

$$\boxed{\text{Full} = 23.55\%, \qquad \text{Incremental} = 23.46\%, \qquad \text{Generational} = 20.93\%}$$

Full and Incremental GC show very similar behaviour because the Full wrapper performs its explicit `LUA_GCCOLLECT` operation before the benchmark rather than establishing a permanent full-collection mode.

Generational GC has the lowest measured GC instruction share. Its execution contains far fewer ordinary propagation and state-machine steps and instead makes substantial use of generation-aware sweeping. This behaviour is consistent with the benchmark, where most allocated objects are short-lived and only a small fraction survive for longer.

Therefore, for this particular allocation-heavy workload, the Generational configuration performs less GC instruction work than the Full and Incremental configurations.

The Callgrind call graphs and raw Callgrind output files used to obtain and verify these results are submitted separately as supporting evidence.